

Lab 07

Structured Assembly Programming

Objectives

1. To understand the purpose and structure of the .data, .code, and .stack segments in organized assembly programs.
2. To transition from flat assembly to structured programming for better readability, maintainability, and scalability.
3. To write and execute simple structured programs in EMU8086 that demonstrate data declaration and character or string display.

Introduction

In the previous lab, programs were written in a **flat assembly format**, where all instructions were placed sequentially without separating data, code, or stack areas. This approach helped introduce the fundamental concepts of assembly such as **registers**, **data movement (MOV instruction)**, and **DOS interrupts (INT 21h)** for performing basic input/output tasks.

While flat assembly is useful for learning the basics, it becomes difficult to manage as programs grow larger or more complex. To overcome this limitation, assembly language supports a **structured programming model** using **segments**.

In this lab, students will learn how to organize their code into three logical sections:

- **.data segment** – for declaring variables and constants.
- **.stack segment** – for allocating temporary storage space used during execution.
- **.code segment** – for writing executable instructions that operate on the data.

This structured approach mirrors how real assemblers like **MASM** handle larger programs and is essential for developing professional, maintainable assembly code. Students will apply this knowledge to write programs that display characters and strings using EMU8086 while adhering to structured coding conventions.

Program 1

```
.model small
.stack 100h

.data
    char db 'A'    ; Character to print

.code
main proc
    mov ax, @data    ; Initialize data segment
    mov ds, ax

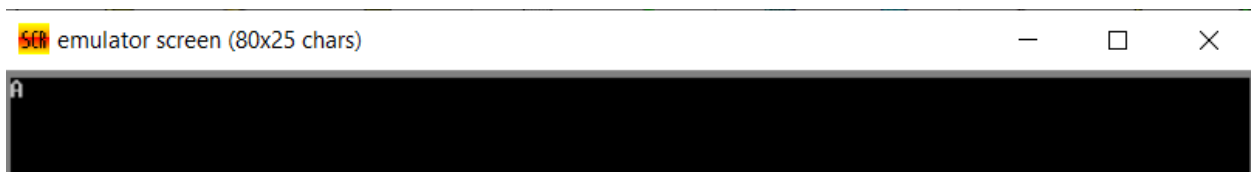
    mov ah, 02h      ; DOS print character function
    mov dl, [char]    ; Load the value from memory
    int 21h          ; Display it

    mov ah, 4Ch      ; Exit program
    int 21h
main endp
end main
```

Explanation:

- model small, .stack 100h – Define memory model and stack space.
- data – Declare variable char holding 'A'.
- Initialize data segment with mov ax, @data and mov ds, ax.
- Set ah = 02h for DOS print function.
- Load character from memory into dl using mov dl, [char].
- Call int 21h to display the character.
- Use mov ah, 4Ch and int 21h to terminate the program.
- mov ax, @data: Loads the starting address of the data segment into the AX register.
- mov ds, ax: Copies the address from AX to DS to activate the data segment for correct data access.

Output:



Relating to C++

```
int main() {  
    cout << "A";  
    return 0;  
}
```

Assembly equivalent

```
main proc  
    mov ah, 02h  
    mov dl, 'A'  
    int 21h  
main endp  
end main
```

Memory Models

Memory Model	Code Segment	Data Segment	Description
TINY	One	One	Both code and data in a single 64KB segment (for .COM files)
SMALL	One	One	One code and one data segment (most common for .EXE programs)
MEDIUM	Many	One	Multiple code segments, one data segment
COMPACT	One	Many	One code segment, multiple data segments
LARGE / HUGE	Many	Many	Multiple code and data segments (for very large programs)

Printing string

```
.model small  
.stack 100h  
  
.data  
    msg db 'Hello, World!$' ; String ending with $  
  
.code  
main proc
```

```
mov ax, @data      ; Initialize data segment
mov ds, ax

mov ah, 09h        ; DOS print string function
lea dx, msg         ; Load address of string
int 21h            ; Display string

mov ah, 4Ch        ; Exit program
int 21h
main endp
end main
```

Output



Explanation

- .model small, .stack 100h – Set memory model and stack.
- msg db 'Hello, World!\$' – Define string (must end with \$).
- mov ax, @data / mov ds, ax – Initialize data segment.
- mov ah, 09h – Select DOS function 9 (print string).
- lea dx, msg – Load address of string into DX.
- int 21h – Display string.
- mov ah, 4Ch / int 21h – Terminate program.

Difference between string and character

Purpose	Instruction	Why
Print a single character	mov dl, [char]	Because DOS function 02h expects the value of the character
Print a string	lea dx, msg	Because DOS function 09h expects the address of the string

Taking character input

```
.model small
.stack 100h

.data
    msg db 'Enter a character: $'

.code
main proc
    mov ax, @data
    mov ds, ax

    ; Display message
    mov ah, 09h
    lea dx, msg
    int 21h

    ; Take input
    mov ah, 08h
    int 21h          ; Character stored in AL

    ; Display the same character
    mov dl, al
    mov ah, 02h
    int 21h

    mov ah, 4Ch
    int 21h
main endp
end main
```

Output:



Explanation:

- **Data Initialization:** Sets up the data segment to access the msg variable.
- **Display Message:** Uses interrupt 21h with function 09h to print "Enter a character:".

- **Input Character:** Calls interrupt 21h with function 08h to take a single keystroke input (without echo).
- **Store Input:** The pressed key is stored automatically in register AL.
- **Display Input:** Moves AL to DL and prints the same character using interrupt 21h, function 02h.
- **Program Exit:** Ends the program gracefully using interrupt 21h, function 4Ch.

Adding two numbers

```
.model small
.stack 100h

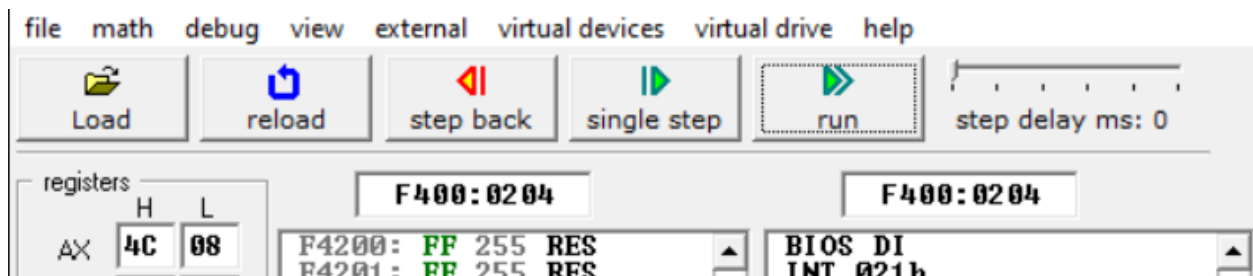
.data
    num1 db 5
    num2 db 3

.code
main proc
    mov ax, @data
    mov ds, ax

    mov al, num1 ; Load first number
    add al, num2 ; Add second number (result in AL)

    mov ah, 4Ch ; Exit program
    int 21h
main endp
end main
```

Output



Arrays

```
.model small
.stack 100h

.data
    arr db 10, 20, 30, 40, 50 ; Array of 5 bytes
```

```

.code
main proc
    mov ax, @data
    mov ds, ax

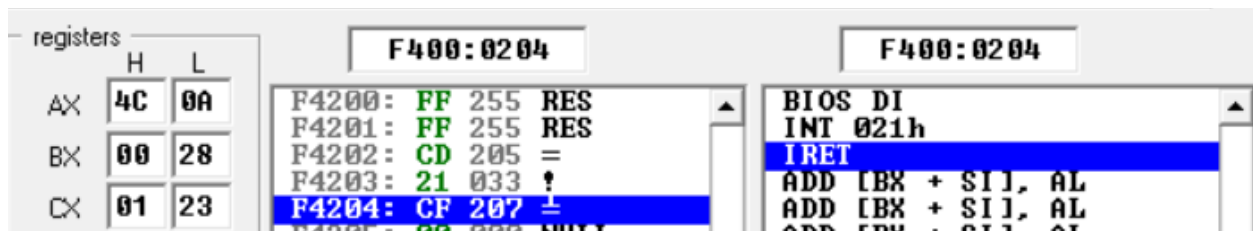
    mov si, 0      ; Index = 0
    mov al, arr[si] ; Load first element (10) into AL

    mov si, 3
    mov bl, arr[si] ; Load 4th element (40) into BL

    mov ah, 4Ch    ; Exit program
    int 21h
main endp
end main

```

Output



Offset not needed.

Case	Memory model	Why offset needed or not
MASM (.model small)	Segmented	DS already points to .data, so arr[si] automatically means DS:arr+si
Flat (NASM/FASM)	Flat	No segment setup — must explicitly use offset or lea to get addresses

Exercise:

Task 1: Take string input and display it on new line.

Task 2: Iterate over an array, calculate the sum and display with a message.

Task 3: Take two integer inputs and print the bigger number with a message.